

Homework Assignment 1

This document contains the write-ups for the four challenges of **Assignment 1**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	Looking Under Rocks	<code>fsuCTF{YouLookedUnderSomeRocksAndFoundAWordlistThisWillBeVeryImportantLaterIREallyHopeYouDidntDoThisManually}</code>	<i>(in Python)</i> • Parse <code>Puzzle</code> to extract line and character coordinates • Load <code>rockyou-45.txt</code> as the reference dictionary • Develop a loop to retrieve specific characters based on coordinates • Reconstruct the flag by concatenating the extracted characters.
Problem 2	Math 2: The Mathening	<code>fsuCTF{n3w_new_n3w_m4th_tm_p4rt_2}</code>	<i>(in Python)</i> • Program the custom operators using bitwise logic • Used <code>pwntools</code> and Regex to automate the input/output cycle
Problem 3	Waiter	<code>fsuCTF{T0_0ff3nd_4_Co0k_47_S34_Is_4_F00ls_Mist4k3}</code>	• Identify nested encoding through hints in the provided notes • Create a script <i>(in Python)</i> to perform recursive Base64 decoding on <code>note3.txt</code> • Monitor the output at each iteration for the <code>fsuCTF{</code> prefix. • Extract the flag when the prefix is found
Problem 4	Endangered Program	<code>fsuCTF{7hi5_I5_7h3_P3rf3c7_3nvir0nm3n7}</code>	• Run <code>cat</code> on the binary to find environment variable dependencies • Identify the <code>VIRTUAL_ENV</code> check used by the program's logic. • Configured the shell environment using <code>export VIRTUAL_ENV=true</code>. • Executed the binary to bypass the check and get the flag.

PROBLEM 1 - Looking Under Rocks

Challenge Description

The challenge provides a file named `Puzzle` containing pairs of numbers and a hint about a famous security breach where the number `45` is significant. We are also directed to the **SecLists** repository.

1. Analysis

First, I examined the contents of the `Puzzle` file:

```
head -n 5 Puzzle
```

The output showed pairs of integers:

```
1027, 5
3, 5
4, 7
...
```

Based on the hint, I searched for files containing "45" in the `seclists` directory:

```
find . -name "*45"
```

I then realized that one of the wordlists containing the number "45" actually came from `rockyou.txt`, a well-known password list originating from a major security breach, as hinted in the challenge description.

```
./Fuzzing/User-Agents/operating-platform/htc-p4550.txt
./Fuzzing/User-Agents/operating-platform/samsung-sch-u450.txt
./Fuzzing/User-Agents/operating-platform/lg-e455g.txt
./Fuzzing/User-Agents/operating-platform/lg-ls450.txt
./Fuzzing/User-Agents/operating-platform/lg-e451g.txt
./Fuzzing/User-Agents/operating-platform/lg-h345.txt
./Fuzzing/User-Agents/operating-platform/sony-kdl-60ex645-digital-tv.txt
./Fuzzing/User-Agents/operating-platform/samsung-sch-i545l.txt
./Fuzzing/User-Agents/operating-platform/lg-h845.txt
./Fuzzing/User-Agents/operating-platform/lg-ms345.txt
./Fuzzing/User-Agents/operating-platform/lg-h445.txt
./Fuzzing/User-Agents/operating-platform/lg-e455.txt
./Fuzzing/User-Agents/operating-platform/lg-e450g.txt
./Fuzzing/User-Agents/operating-platform/lg-k450.txt
./Fuzzing/User-Agents/operating-platform/samsung-sch-i545pp.txt
./Fuzzing/User-Agents/operating-platform/lg-e450.txt
./Fuzzing/User-Agents/operating-platform/lg-e455f.txt
./Fuzzing/User-Agents/operating-platform/sony-kdl-46ex645-digital-tv.txt
./Fuzzing/User-Agents/operating-platform/sony-kdl-50ex645-digital-tv.txt
./Fuzzing/User-Agents/operating-platform/samsung-sch-i545.txt
./Fuzzing/User-Agents/operating-platform/htc-p3450.txt
./Fuzzing/User-Agents/operating-platform/lg-e445.txt
./Fuzzing/User-Agents/operating-platform/lg-lgms450.txt
./Fuzzing/User-Agents/operating-platform/samsung-gt-7245.txt
./Fuzzing/User-Agents/operating-platform/lumia-645.txt
./Fuzzing/User-Agents/operating-platform/lg-x145.txt
./Fuzzing/User-Agents/operating-platform/lg-lgmp450.txt
./Fuzzing/User-Agents/operating-platform/htc-6545lvw.txt
./Fuzzing/User-Agents/operating-platform/lg-tp450.txt
./Fuzzing/User-Agents/operating-platform/lg-l45c.txt
./Fuzzing/User-Agents/operating-platform/lg-e450f.txt
./Fuzzing/User-Agents/operating-platform/lg-vs450pp.txt
./Fuzzing/User-Agents/operating-platform/lg-e450b.txt
./Fuzzing/User-Agents/operating-platform/lg-mp450.txt
./Fuzzing/User-Agents/operating-platform/samsung-sgh-t459.txt
./Fuzzing/User-Agents/operating-platform/lg-h450.txt
./Passwords/Leaked-Databases/rockyou-45.txt
```

2. Solution Strategy

Looking at the pairs of integers in the `Puzzle` file, and considering that the second numbers are small (0–7) while the first numbers match the scale of a wordlist, a **Book Cipher** approach seems logical. In this context, each pair of numbers represents coordinates:

- **First number:** the line index in `rockyou-45.txt` (Starting from 0)
- **Second number:** the character index within that line (Starting from 0)

I developed a Python script to automate the extraction of characters from the wordlist.

```

# Read coordinates from the Puzzle file
puzzle_file = open('Puzzle', 'r')
coordinates = []
for line in puzzle_file:
    parts = line.split(',')

    row = int(parts[0])    #line cord
    column = int(parts[1]) #letter cord

    coordinates.append((row, column))
puzzle_file.close()

# Read the dictionary file rockyou-45.txt
dict_file = open('rockyou-45.txt', 'r')
lines = dict_file.readlines()
dict_file.close()

# Build the flag
flag = ""
for coord in coordinates:
    row_index = coord[0]
    char_index = coord[1]

    # Get the character from the line
    target_line = lines[row_index]
    character = target_line[char_index]

    # Append the character to our flag string
    flag = flag + character

print("fsuCTF{" + flag + "}")

```

3. Execution and Flag

Running the script against the provided files shows the decoded message.

```

(ende@ende)-[~/.../CTF/labs/Python/Looking Under Rocks]
$ python3 rocks.py
fsuCTF{YouLookedUnderSomeRocksAndFoundAWordlistThisWillBeVeryImportantLaterIReallyHopeYouDidntDoThisManually}

```

Flag:

```

fsuCTF{YouLookedUnderSomeRocksAndFoundAWordlistThisWillBeVeryImportantLaterIReallyHopeYouDidntDoThisManually}

```

PROBLEM 2 - Math 2: The Mathening

Challenge Description

The challenge introduces "Daniel's New-Math™," a set of custom mathematical operations based on bitwise logic. We are provided with a specification sheet and a remote server (NC) that requires us to solve 250 to

get the flag.

1. Analysis

The core of the challenge lies in correctly implementing three custom operations using **signed 64-bit integers**. In Python, this is tricky because integers have arbitrary precision, so we must manually handle overflows and bitwise masks to simulate a 64-bit environment.

- **1. (@) A-Transform**
 - **Logic:** `NOT(A) AND NOT(B)`, then `XOR` the result with `B`.
 - **Formula:** `((~A) & (~B)) ^ B`
- **2. (#) Middle Mash**
 - **Logic:** `NOT(A) OR NOT(B)`, shift right by 2, then `XOR` with `A` shifted left by 2.
 - **Formula:** `((~A | ~B) >> 2) ^ (A << 2)`
- **3. (?) Merge**
 - **Logic:** Interleaving bits. Take every other bit of A (starting from MSB) and every second-highest bit of B.
 - **Implementation:** Using hexadecimal masks:
 - Mask A: `0xAAAAAAAAAAAAAAAA` (101010...)
 - Mask B: `0x5555555555555555` (010101...)

2. Solution Strategy

To automate the process, I used the `pwntools` library in Python. The script used to solve the problem follows the following steps:

1. **Connection:** Establishes a TCP socket with the server using `pwntools`.
2. **Parsing:** Uses `Regex` to instantly extract the numbers (A, B) and the operator from the server's message.
3. **Bitwise Logic:** Executes the "New-Math" operations.
4. **Signing:** Converts the unsigned 64-bit result into a **signed decimal** (two's complement) required by the server.
5. **Automation:** Loops 250 times
6. **Flag Retrieval:** Collects and prints the final flag after all rounds are cleared

```
from pwn import *
import re

HOST = 'ctf.cs.fsu.edu'
PORT = 65002
MASK = 0xFFFFFFFFFFFFFFFF

def to_signed_64(val):
    # Convierte un valor de 64 bits a su representación con signo
    val = val & MASK
    if val > 0x7FFFFFFFFFFFFFFF:
        return val - 0x10000000000000000
    return val
```

```

def a_transform(a, b):
    # (@) A-Transform: ( $\sim A \& \sim B$ ) ^ B
    not_a = ( $\sim a$ ) & MASK
    not_b = ( $\sim b$ ) & MASK
    res_and = (not_a & not_b) & MASK
    raw_ans = (res_and ^ b) & MASK
    return str(to_signed_64(raw_ans))

def middle_mash(a, b):
    # (#) Middle Mash: ( $(\sim A \mid \sim B) \gg 2$ ) ^ ( $A \ll 2$ )
    ans = (( $\sim a \mid \sim b$ ) >> 2) ^ (a << 2)
    return str(ans)

def merge(a, b):
    # (?) Merge: Bits intercalados.
    mask_a = 0xAAAAAAAAAAAAAAAA
    mask_b = 0x5555555555555555
    res_a = a & mask_a
    res_b = b & mask_b
    raw_ans = (res_a | res_b) & MASK
    return str(raw_ans)

def solve():
    # Connection to Server
    r = remote(HOST, PORT)

    # Skip Initial Banner
    print(r.recvuntil(b"enough.").decode())

    for i in range(250):
        # Read question until '=' character
        data = r.recvuntil(b" = ").decode()
        print(f"{data.strip()}", end=" ")

        # Regex to extract A, Operator and B
        match = re.search(r"(-?\d+)\s+([@#?])\s+(-?\d+)", data)
        if not match:
            break

        A = int(match.group(1))
        op = match.group(2)
        B = int(match.group(3))

        # Seleccionamos la operación según el símbolo
        if op == '@':
            payload = a_transform(A, B)
        elif op == '#':
            payload = middle_mash(A, B)
        elif op == '?':
            payload = merge(A, B)

        # Send Response
        r.sendline(payload.encode())

```

```

    print(f"-> Sent: {payload}")

# Read flag
flag = r.recvall(timeout=5).decode()
print("=====")
print(flag)

solve()

```

3. Execution and Flag

When executing the script, all 250 operations are successfully completed, and the flag is obtained.

```

[247] -1606988152352829371 @ -2086931601272203214 = → Sent: -626638108504195142
[248] -1868238156102575738 @ -449473279935088630 = → Sent: -436999801520554373
[249] -8265011323232845979 ? 3957364821684700143 = → Sent: 11261746733405629285
[+] Receiving all data: Done (43B)
[*] Closed connection to ctf.cs.fsu.edu port 65002
Thanks! fsuCTF{n3w_new_n3w_m4th_tm_p4rt_2}

```

Flag: `fsuCTF{n3w_new_n3w_m4th_tm_p4rt_2}`

PROBLEM 3 - Waiter

Challenge Description

In this challenge we encounter a bulletin board with three notes. The goal is to extract a hidden flag from a massive data file named `note3.txt`.

1. Analysis

The key to the challenge lies in the hints found in the first two notes:

- **Note 1 (Patty):** "Someone needs to start put expiration **LABELS** on the food or I'm going to **JUMP** ship."
- **Note 2 (Carne):** "Make more soup **BASE**, we are running out. We should probably assign this to someone specific so they can master the process and **repeat it quickly**."

Interpretation:

- **BASE:** A direct hint towards **Base64** encoding
- **Repeat it quickly:** Suggests that the encoding is **recursive** (layers within layers) and requires automation
- **LABELS:** Might refer to the "padding" characters (`=`) used in Base64 to align data

The content of `note3.txt`, a single line of 100,000 characters, confirms that the data might be heavily layered and be too large for manual decoding.

2. Solution Strategy

To solve this, we need a script that repeatedly decodes the content of `note3.txt` using Base64 until the flag format `fsuCTF{...}` is detected.

```
import base64

def solve():
    # read note3.txt
    f = open('note3.txt', 'r')
    data = f.read().strip()
    f.close()

    iteration = 0
    while True:
        # Decode Base64
        decoded_bytes = base64.b64decode(data)
        data = decoded_bytes.decode('utf-8')
        iteration += 1

        # Buscar la flag
        if "fsuCTF{" in data:
            print("=====")
            print(f"Flag found in layer: {iteration}")
            print(f"Flag: {data}")
            print("=====")
            break

solve()
```

3. Execution and Flag

The script is executed to recursively decode the multiple Base64 layers within `note3.txt` until the `fsuCTF{...}` flag is found in layer 26.

```
$ python3 waiter.py
=====
Flag found in layer: 26
Flag: fsuCTF{T0_0ff3nd_4_Co0k_47_S34_Is_4_F001s_Mist4k3}
=====
```

Flag: `fsuCTF{T0_0ff3nd_4_Co0k_47_S34_Is_4_F001s_Mist4k3}`

PROBLEM 4 - Endangered Program

Challenge Description

The challenge provides a 64-bit Linux executable named `EndangeredProgram`. The prompt indicates that the program "only survives in certain environments" and will provide a "gift" (the flag) if run in its preferred environment.

1. Analysis

```
$ file EndangeredProgram
EndangeredProgram: ELF 64-bit LSB pie executable, x86_64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=c806995def10d610453a5c3aedf3002d7efada71, for GNU/Linux 3.2.0, not stripped
```

After inspecting the binary with the `file` command, we confirm it is an **ELF 64-bit LSB pie executable**. Since we were instructed not to reverse engineer it, we perform a basic string analysis with `cat`.

[illegible]

The output reveals several interesting strings:

- `VIRTUAL_ENV`: A common environment variable used by Python to indicate an active virtual environment.
- `The endangered program loves this environment.`: The success message.
- `The endangered program hates this environment.`: The failure message.
- `getenv`: A standard C library function used to retrieve the value of environment variables.

This implies the program checks for the existence of the `VIRTUAL_ENV` variable.

2. Solution Strategy

The strategy is to simulate the "environment" the program prefers.

Since the program simply checks if `VIRTUAL_ENV` is set, we can solve this by:

1. Defining the `VIRTUAL_ENV` variable with a dummy value.
2. Executing the binary within that same shell context.

3. Execution and Flag

To solve the challenge, we set the required environment variable and run the program:

```
# Set the environment variable the program is looking for
export VIRTUAL_ENV=true

# Execute the program
./EndangeredProgram
```

After the execution in the proper environment we get the flag.

```
└─$ ./EndangeredProgram
The endangered program loves this environment.
It is so happy that it has decided to gift you a flag.
fsuCTF{7hi5_I5_7h3_P3rf3c7_3nvir0nm3n7}
```

Flag: `fsuCTF{7hi5_I5_7h3_P3rf3c7_3nvir0nm3n7}`